SatsPath × Arkade — Technical Integration Plan (Production / v2)

Goal: Fully native, self-sovereign SatsPath stack in Arkade wallet (mobile + desktop) with zero trusted backend. UniFFI bindings for Kotlin (Android), Swift (iOS), TypeScript (React Native / Tauri / PWA).

Architecture:

```
Arkade Wallet (Kotlin/Swift/TS)
    ↓ UniFFI
satspath-core + satspath-router (Rust cdylib)
      Resolver Chain (local → BIP353 → HTTPS → Nostr → P2P)
      Profile Crypto (secp256k1 sign/verify)
      Router (fee-aware rail selection)
      QR/Payment Payload Builder
    ↓ Execution Adapters (platform-specific)
LDK (Lightning) / Arkade SDK (Ark) / BDK (On-chain)
```

---

1. Production Requirements (vs MVP)

Area	MVP	Production
Resolver	TS fetch in PWA	Rust async in core (tokio) — no JS fetch dependency
Crypto	WASM (secp256k1 only)	Native Rust (secp256k1 + schnorrkel) via UniFFI
Router	TS port (~300 LOC)	Native Rust (full scoring, urgency, split payments)
Identity	LocalStorage profile	Encrypted SQLCipher/Keychain/Keystore + BIP-39 seed backup
P2P	Optional satspathd sidecar	Embedded Hyperswarm via UniFFI (or libp2p)
Ark	Testnet preview	Mainnet ready (covenant-based when available)
BOLT12		Offer-based async payments
Silent Payments		BIP-352 receive
Split Payments		Multi-rail single payment
Key Rotation		Spec §29
Platform Verification	Invite only	DKIM / OAuth / Nostr NIP-42

---

2. Rust Refactor for UniFFI

2.1 Crate Restructuring

```
satspath/
  crates/
      satspath-core/          # → Keep, make `cdylib` + `uniffi` feature
      satspath-router/        # → Keep, make `cdylib` + `uniffi` feature
      satspath-ffi/           # NEW: UniFFI definitions (UDL) + glue
          satspath.udl
          build.rs
          src/
              resolver.rs     # ResolverChain implementation
              router.rs       # Router FFI wrapper
```

```
            profile.rs      # Profile management FFI
            crypto.rs       # Crypto FFI wrapper
      satspath-wasm/         # Keep for web-only fallback
```

2.2 satspath-core Changes

Cargo.toml:

```toml
[lib]
crate-type = ["rlib", "cdylib"]  # cdylib for UniFFI

[features]
default = []
uniffi = ["uniffi", "tokio/rt-multi-thread", "reqwest/native-tls"]

[dependencies]
uniffi = { version = "0.28", optional = true }
tokio = { version = "1", features = ["rt-multi-thread", "macros"], optional = true }
reqwest = { version = "0.12", features = ["json", "native-tls"], optional = true }
# ... existing deps
```

Expose async trait for UniFFI (src/resolver.rs):

```rust
use uniffi::async_trait;

#[async_trait]
pub trait ProfileResolver: Send + Sync {
    async fn resolve_alias(&self, alias: &str) -> Result<SignedPaymentProfile, SatsPathError>;
}

// Implement for each resolver: LocalRegistry, Bip353, HttpWellKnown, NostrNip05, P2P
```

2.3 UDL Definitions (satspath-ffi/satspath.udl)

```
namespace satspath;

interface ProfileResolver {
    Promise<SignedPaymentProfile> resolveAlias(string alias);
}

enum PaymentMethodType { Onchain, Lightning, Ark };

record OnchainMethod {
    string label;
    string network;  // "mainnet" | "testnet" | "regtest"
    string? address;
    string? silentPaymentPubkey;
    string? pubkeyHint;
    string? descriptorHint;
    sequence<string> addressList;
};

record LightningMethod {
    string label;
    string? lightningAddress;
    string? lnurl;
```

```
    string? bolt12;
    string? receiverPubkey;
};

record ArkMethod {
    string label;
    string server;
    string pubkey;
    string? vtxoPointer;
    string? opaqueUri;
    ArkOwnershipProof? proof;
    i64? expiresAt;
};

union PaymentMethod { OnchainMethod | LightningMethod | ArkMethod };

record PaymentProfile {
    string alias;
    string identityPubkey;
    sequence<PaymentMethod> methods;
    i64 updatedAt;
    i64? expiresAt;
    u64? sequence;
    sequence<string> preferences;
    string? nonce;
    KeyRotation? rotation;
    sequence<MethodVerification> methodVerifications;
};

record SignedPaymentProfile {
    PaymentProfile profile;
    string signature;  // hex schnorr
};

record QuoteRequest {
    string recipient;
    u64 amountSats;
    SignedPaymentProfile signedProfile;
    string urgency;  // "low" | "normal" | "high"
    u64? maxFeeSats;
    f64? maxFeePercent;
};

enum ExecutionMode { Preview, MainnetPreview, TestnetExperimental, ManualWallet };

record RouteQuote {
    PaymentMethod selectedMethod;
    u64 estimatedFeeSats;
    string estimatedConfirmation;
    string reason;
    ExecutionMode execution;
    string walletHint;
};
```

```
record FeeEstimate {
    u64 fastestFee;
    u64 halfHourFee;
    u64 hourFee;
    u64 economyFee;
    u64 minimumFee;
};

interface Router {
    Promise<RouteQuote> selectRoute(QuoteRequest request, FeeEstimate fees);
    Promise<FeeEstimate> fetchFeeEstimate();
    string buildQrPayload(PaymentMethod method, u64 amountSats);
};

enum QuoteStatus { Ok, NotRegistered, NoRoute, InvalidSignature };

record QuoteRecipient {
    string alias;
    boolean verified;
    boolean profileSignatureVerified;
    boolean identifierVerified;
    string identifierVerification;
    string fingerprint;
};

record Invite {
    string aliasHash;
    u64 amountSats;
    i64 createdAt;
    i64 expiresAt;
    string claimUrl;
    string warning;
    string? senderSignature;
    string? senderPubkey;
};

union QuoteResponse {
    record { QuoteRecipient recipient; PaymentMethod selectedMethod; u64 feeSats; string eta; string rea
    record { Invite invite; },
    record { string reason; },
    record { QuoteRecipient recipient; }
};

interface Satspath {
    // High-level: resolve + verify + route + build payload
    Promise<QuoteResponse> quote(string recipient, u64 amountSats);

    // Lower-level building blocks
    Promise<SignedPaymentProfile> resolve(string alias);
    boolean verifyProfile(SignedPaymentProfile profile);
    Promise<RouteQuote> route(QuoteRequest request);

    // Identity management
    Promise<Identity> createIdentity();
```

```
    Promise<void> saveProfile(SignedPaymentProfile profile);
    Promise<SignedPaymentProfile?> loadProfile(string alias);

    // P2P (optional)
    Promise<void> startP2pBridge(string profilePath);
    void stopP2pBridge();
};

record Identity {
    string pubkeyHex;
    string secretKeyPath;  // encrypted, platform keystore
    string fingerprint;
};
```

2.4 FFI Implementation (satspath-ffi/src/)

```rust
// satspath-ffi/src/resolver.rs
use uniffi::async_trait;
use satspath_core::resolver::{ChainResolver, ProfileResolver as CoreResolver};
use satspath_core::resolvers::{bip353::Bip353Resolver, http::HttpResolver, nostr::NostrResolver};

pub struct ResolverChain {
    inner: ChainResolver,
}

#[uniffi::export(async_runtime = "tokio")]
impl ResolverChain {
    #[uniffi::constructor]
    pub fn new() -> Self {
        let mut chain = ChainResolver::new();
        chain = chain.push(LocalRegistryResolver::new()); // impl CoreResolver
        chain = chain.push(Bip353Resolver::new());
        chain = chain.push(HttpResolver::new());
        chain = chain.push(NostrResolver::new());
        // P2P optional
        Self { inner: chain }
    }

    pub async fn resolve_alias(&self, alias: String) -> Result<SignedPaymentProfile, SatsPathError> {
        self.inner.resolve_alias(&alias).await
    }
}

// satspath-ffi/src/router.rs
use satspath_router::{select_route, select_route_with_fees, FeeEstimate, RouteRequest, build_qr_payload

#[uniffi::export(async_runtime = "tokio")]
pub async fn select_route_ffi(request: QuoteRequest, fees: FeeEstimate) -> Result<RouteQuote, SatsPathE
    let req = RouteRequest {
        alias: request.recipient,
        amount_sats: request.amount_sats,
        signed_profile: request.signed_profile.into(),
        urgency: parse_urgency(request.urgency),
        max_fee_sats: request.max_fee_sats,
```

```
        max_fee_percent: request.max_fee_percent,
    };
    let quote = select_route_with_fees(&req, &fees)?;
    Ok(quote.into())
}

#[uniffi::export(async_runtime = "tokio")]
pub async fn fetch_fee_estimate_ffi() -> Result<FeeEstimate, SatsPathError> {
    satspath_router::fees::fetch_fee_estimate().await.map_err(Into::into)
}

// satspath-ffi/src/profile.rs
#[uniffi::export]
pub fn verify_profile(profile: SignedPaymentProfile) -> bool {
    satspath_core::crypto::verify_signed_profile(&profile.into()).unwrap_or(false)
}

#[uniffi::export(async_runtime = "tokio")]
pub async fn quote(recipient: String, amount_sats: u64) -> QuoteResponse {
    let resolver = ResolverChain::new();
    satspath_router::quote_with_resolver(&resolver, &recipient, amount_sats).await.into()
}
```

2.5 Build Pipeline

```
# 1. Generate bindings
cd satspath-ffi
cargo build --release --features uniffi
cargo run --features uniffi --bin uniffi-bindgen generate \
  --library target/release/libsatspath_ffi.so \
  --language kotlin --out-dir ../../arkade-wallet/android/satspath/src/main/java
cargo run --features uniffi --bin uniffi-bindgen generate \
  --library target/release/libsatspath_ffi.so \
  --language swift --out-dir ../../arkade-wallet/ios/Satspath
cargo run --features uniffi --bin uniffi-bindgen generate \
  --library target/release/libsatspath_ffi.so \
  --language typescript --out-dir ../../arkade-wallet/src/satspath

# 2. Android: copy .so to jniLibs
# 3. iOS: embed .framework in Xcode
# 4. TS: npm install @satspath/core (or file:../satspath)
```

---

3. Platform-Specific Integration

3.1 Android (Kotlin + LDK + Arkade SDK + BDK)

Gradle (`android/app/build.gradle.kts`):

```
dependencies {
    implementation("org.ldk:ldk-android:0.0.120")  // or Breez SDK
    implementation("org.bitcoindevkit:bdk-android:1.0.0")
    implementation(project(":satspath"))  // UniFFI generated module
    // Arkade SDK as local module or maven
}
```

Identity Storage (android/satspath/src/main/java/com/satspath/IdentityManager.kt):

```kotlin
class IdentityManager @Inject constructor(
    @Assisted private val context: Context
) {
    private val keystore = KeyStore.getInstance("AndroidKeyStore")
    private val prefs = context.getSharedPreferences("satspath", Context.MODE_PRIVATE)

    fun createIdentity(): Identity {
        val kp = Satspath.createIdentity()  // UniFFI call
        // Store secret in Keystore, pubkey in SharedPreferences
        storeSecret(kp.secretKeyPath)
        prefs.edit().putString("pubkey", kp.pubkeyHex).apply()
        return kp
    }

    fun signProfile(profile: PaymentProfile): SignedPaymentProfile {
        val secret = loadSecret()  // from Keystore
        return Satspath.signProfile(profile, secret)
    }
}
```

Send Flow (android/ui/send/SendViewModel.kt):

```kotlin
class SendViewModel @ViewModelInject constructor(
    private val satspath: Satspath,
    private val ldk: LdkManager,
    private val arkade: ArkadeSdk,
    private val bdk: BdkManager
) : ViewModel() {

    fun getQuote(alias: String, amountSats: Long) = viewModelScope.launch {
        val response = satspath.quote(alias, amountSats)
        _quoteState.value = response
    }

    fun executePayment(response: QuoteResponse) = viewModelScope.launch {
        when (response.status) {
            QuoteStatus.Ok -> {
                val method = response.selectedMethod
                when (method) {
                    is PaymentMethod.LightningMethod -> {
                        val invoice = if (response.qr.startsWith("lnbc")) response.qr
                            else ldk.fetchInvoice(response.qr, response.feeSats)
                        ldk.sendPayment(invoice)
                    }
                    is PaymentMethod.OnchainMethod -> {
                        bdk.sendPayment(response.qr)  // BIP-21 URI
                    }
                    is PaymentMethod.ArkMethod -> {
                        arkade.sendPayment(response.qr)  // ark: URI
                    }
                }
            }
            QuoteStatus.NotRegistered -> showInviteFlow(response.invite)
```

```
        }
    }
}
```

3.2 iOS (Swift + LDK + Arkade SDK + BDK)

Package.swift:

```
dependencies: [
    .package(url: "https://github.com/lightningdevkit/ldk-swift", from: "0.1.0"),
    .package(url: "https://github.com/bitcoindevkit/bdk-swift", from: "1.0.0"),
    // Satspath framework embedded in Xcode
]
```

Identity (SatspathIdentityManager.swift):

```
class SatspathIdentityManager {
    let keychain = Keychain(service: "com.arkade.satspath")

    func createIdentity() throws -> Identity {
        let identity = try Satspath.createIdentity()  // UniFFI
        try keychain.set(identity.secretKeyPath, key: "satspath_secret_\(identity.fingerprint)")
        return identity
    }

    func signProfile(_ profile: PaymentProfile) throws -> SignedPaymentProfile {
        let secret = try keychain.get("satspath_secret_\(currentFingerprint)")
        return try Satspath.signProfile(profile, secret)
    }
}
```

Send Flow (SendViewModel.swift):

```
@MainActor
class SendViewModel: ObservableObject {
    @Published var quote: QuoteResponse?

    func getQuote(alias: String, amount: UInt64) async {
        quote = try await Satspath.shared.quote(alias, amount)
    }

    func pay() async throws {
        guard case .ok(let data) = quote?.status else { return }
        switch data.selectedMethod {
        case .lightning(let method):
            let invoice = data.qr.hasPrefix("lnbc") ? data.qr : try await LDKManager.shared.fetchInvoic
            try await LDKManager.shared.sendPayment(invoice)
        case .onchain(let method):
            try await BDKManager.shared.sendPayment(data.qr)
        case .ark(let method):
            try await ArkadeSDK.shared.sendPayment(data.qr)
        }
    }
}
```

3.3 React Native / Tauri / PWA (TypeScript)

npm dependency: `@satspath/core` (generated from UniFFI TS bindings)

```typescript
// packages/satspath-core/src/index.ts (generated)
export interface Satspath {
  quote(recipient: string, amountSats: bigint): Promise<QuoteResponse>;
  resolve(alias: string): Promise<SignedPaymentProfile>;
  verifyProfile(profile: SignedPaymentProfile): boolean;
  createIdentity(): Promise<Identity>;
  // ...
}

export const satspath: Satspath; // WASM fallback if native not available
```

Usage (same as MVP but native):

```typescript
import { satspath } from "@satspath/core";

const response = await satspath.quote("alice@example.com", 21000n);
if (response.status === "ok") {
  switch (response.selected_method.type) {
    case "Lightning":
      await ldkSend(response.qr);
      break;
    case "Onchain":
      await bdkSend(response.qr);
      break;
    case "Ark":
      await arkadeSend(response.qr);
      break;
  }
}
```

---

4. Ark Mainnet Readiness (Production Blockers)

Blocker	Status	Solution
Covenant opcodes (CTV/CSFS)	Not on mainnet	Use connector outputs + pre-signed txs (current Ark design)
Ark server trust	ASP can steal if malicious	Client-side VTXO verification (Arkade SDK Tier 1-3) — already done
Unilateral exit UX	Complex	Arkade SDK `sovereignStorage` + auto-broadcast on expiry
Liquidity	Limited	Boltz submarine swaps for on-ramp/off-ramp
Fee estimation	Ark server sets	Router queries ASP `/fee` endpoint, falls back to mempool

Router Ark Logic (enhance `ark_routes.rs`):

```rust
pub async fn plan_ark_route_mainnet(
    sender: &SenderCapabilities,
```

```
    receiver: &SignedPaymentProfile,
    asp_fees: &AspFeeInfo,
) -> Option<ArkRoutePlan> {
    // Verify ASP fee policy is reasonable
    // Check receiver VTXO expiry > 144 blocks (safety margin)
    // Ensure sender has VTXOs on same ASP or can swap via Boltz
    // Return ArkToArk / ArkToLightning / ArkToOnchain with real fee estimates
}
```

---

5. BOLT12 / Silent Payments / Split Payments (v2.1+)

5.1 BOLT12 Offers

```
// satspath-core/src/profile.rs - extend PaymentMethod::Lightning
Lightning {
    // ... existing
    bolt12: Option<String>,  // offer: lno1...
    // Add: offer fetching + invoice_request flow
}
```

Router: if `bolt12` present → fetch offer → send `invoice_request` → pay invoice.

5.2 Silent Payments (BIP-352)

```
Onchain {
    silent_payment_pubkey: Option<String>,  // sp1q...
    // Payer derives: address = hash(sp_pubkey + shared_secret)
}
```

Router: if `silent_payment_pubkey` present → derive unique address per payment.

5.3 Split Payments

```
// New route type
enum RouteQuote {
    Single(RouteQuote),
    Split(Vec<SplitRoute>),  // { method, amount_sats, fee_sats } summing to total
}
```

Scoring: minimize total fee, respect per-rail limits.

---

6. Key Rotation & Profile Revocation (Spec §29)

UDL Addition:

```
record KeyRotation {
    string newIdentityPubkey;
    i64 rotationTime;
    string previousSignature;  // sig by old key over new key
};

interface Satspath {
    Promise<SignedPaymentProfile> rotateKey(string alias, string newPubkeyHex);
```

```
    boolean verifyRotation(SignedPaymentProfile oldProfile, SignedPaymentProfile newProfile);
}
```

Flow: 1. User generates new identity key (in Secure Enclave/Keystore) 2. Sign `rotation` object with old key 3. Publish new profile with `rotation` field + both signatures 4. Resolvers verify chain: `old_sig` verifies `rotation` + `new_sig` verifies `profile`

---

7. Platform Verification (Email/Domain Ownership)

Current: Invite flow only (no proof)

Production: | Method | Verification | UDL | |———|————–|—| | **DKIM** | User forwards verification email → wallet verifies DKIM sig | `EmailVerifier` interface | | **Nostr NIP-42** | Relay challenges user to sign event | `NostrVerifier` | | **DNS** | User adds TXT record → resolver checks | `DnsVerifier` (BIP-353) | | **OAuth** | Wallet opens auth flow → gets signed attestation | `OAuthVerifier` |

Profile Field:

```
record MethodVerification {
    string methodDescriptor;  // PaymentMethod::ownership_descriptor()
    string proofType;         // "dkim" | "nostr" | "dns" | "oauth"
    string proofData;         // JSON attestation
    i64 verifiedAt;
}
```

---

8. P2P Transport (Optional but Sovereign)

Current: `satspathd` spawns Node.js Holepunch bridge

Production: Embed Hyperswarm in Rust via `hyperdriver` (or `libp2p`)

```rust
// satspath-core/src/p2p.rs
pub struct P2pResolver {
    swarm: HyperSwarm,
    topic: Topic,
}

#[async_trait]
impl ProfileResolver for P2pResolver {
    async fn resolve_alias(&self, alias: &str) -> Result<SignedPaymentProfile, SatsPathError> {
        let topic = derive_topic(alias);  // SHA256("satspath-profile:" + alias)
        let mut stream = self.swarm.join(topic).await?;
        // Request profile via DHT
        let profile = stream.request(alias).await?;
        verify_signed_profile(&profile)?;
        Ok(profile)
    }
}
```

UniFFI: Expose `startP2p()` / `stopP2p()` — mobile runs swarm in background.

---

9. Testing & CI/CD (Production)

9.1 Rust Layer

```
# .github/workflows/rust.yml
- cargo test --workspace --features uniffi
- cargo clippy --workspace --features uniffi -- -D warnings
- cargo fmt --check
- cargo audit
```

9.2 UniFFI Bindings Tests

```
# Kotlin
cd android && ./gradlew test

# Swift
cd ios && xcodebuild test -scheme SatspathTests

# TypeScript
cd sdk/ts && npm test
```

9.3 Integration Tests (Testnet)

```
// tests/integration/mainnet_preview.rs
#[tokio::test]
async fn mainnet_preview_quote() {
    let satspath = Satspath::new();
    let quote = satspath.quote("rodrigo@satspath.dev", 100_000).await;
    assert!(matches!(quote.status, QuoteStatus::Ok));
    assert!(quote.qr.starts_with("bitcoin:") || quote.qr.starts_with("lnbc"));
}
```

9.4 Fuzz Testing

```
cargo fuzz run profile_parsing
cargo fuzz run router_scoring
```

---

10. File Tree (Production Additions)

```
satspath/
  crates/
      satspath-core/
          Cargo.toml          # +uniffi feature, cdylib
          src/
              p2p.rs          # NEW: embedded Hyperswarm
              rotation.rs     # ENHANCE: key rotation logic
              verification.rs # NEW: DKIM/Nostr/DNS/OAuth verifiers
      satspath-router/
          src/
              bolt12.rs       # NEW: offer → invoice_request flow
              silent.rs          silent_payments.rs   # NEW: BIP-352 address derivation
          split.rs           # ENHANCE: multi-rail routing
      satspath-ffi/          # NEW: UniFFI layer
          Cargo.toml
```

```
        build.rs
        satspath.udl
        src/
            lib.rs
            resolver.rs
            router.rs
            profile.rs
            identity.rs
            p2p.rs
    satspath-wasm/          # Keep for web fallback
sdk/
    kotlin/                 # Generated by UniFFI
    swift/                  # Generated by UniFFI
    typescript/             # Generated by UniFFI
arkade-wallet/
    android/
        satspath/           # UniFFI Kotlin module
    ios/
        Satspath/           # UniFFI Swift module
    src/
        satspath/           # UniFFI TS module
docs/
    TECHNICAL_PLAN_MVP.md
    TECHNICAL_PLAN_PRODUCTION.md
```

---

11. Timeline (Production)

Phase	Weeks	Deliverable
0. Prep	1	Enable `uniffi` feature, `cdylib`, fix WASM-incompatible deps
1. FFI Layer	3	`satspath-ffi` crate + UDL + all bindings (Kotlin/Swift/TS)
2. Resolver Chain	2	Port all resolvers to async Rust trait, remove TS resolvers
3. Router Native	2	Full router in Rust (scoring, urgency, split, BOLT12, silent)
4. Identity & Storage	2	Encrypted keystore (Keystore/Keychain/SQLCipher) + BIP-39 backup
5. P2P Embedded	3	`hyperdriver` integration, background swarm on mobile
6. Platform Verification	2	DKIM + NIP-42 + DNS verifiers
7. Ark Mainnet	3	Covenant-ready ASP integration, fee estimation, liquidity
8. Mobile Integration	4	Arkade Android/iOS wiring (LDK, BDK, Arkade SDK)
9. Auditing & Hardening	3	Security audit, fuzzing, testnet soak, mainnet preview
Total	~25 weeks	**Production-ready Arkade + SatsPath**

---

12. Definition of Done (Production)

- ☐ `cargo build --release --features uniffi` produces `.so`/`.dylib`/`.dll` + bindings for 3 targets
- ☐ Android app: `satspath.quote("alice@example.com", 100_000)` returns in <500ms (no network on cached profile)
- ☐ iOS app: same, using Swift UniFFI module
- ☐ React Native/Tauri: same, using TS bindings
- ☐ Key rotation: user rotates key → new profile verifies on all resolvers
- ☐ P2P: profile resolves via Hyperswarm without HTTPS/DNS
- ☐ Ark mainnet: send/receive VTXO on mainnet ASP, sovereign exit tested
- ☐ BOLT12: pay offer → invoice_request → invoice → pay
- ☐ Silent Payments: derive unique address per payment from `sp1q...`
- ☐ Split Payments: single 500k sat payment splits 300k LN + 200k on-chain
- ☐ DKIM verification: user proves `alice@gmail.com` ownership
- ☐ All integration tests pass on testnet (signet + mutinynet)
- ☐ Security audit completed (crypto, memory safety, supply chain)

---

13. Risk Register (Production)

Risk	Likelihood	Impact	Mitigation
UniFFI async trait limitations	Medium	High	Use `uniffi::async_trait` + `tokio` runtime; test early
Mobile binary size (Rust + deps)	High	Medium	`strip`, `lto`, feature-gate optional modules (P2P, Ark)
Ark mainnet covenant delay	High	High	Ship with connector-output design; upgrade when CTV activates
Platform keystore differences	Medium	Medium	Abstract `SecureStorage` trait per platform
Nostr relay censorship	Low	Medium	Multi-relay fallback; P2P as ultimate fallback
Regulatory (KYC/AML on identity)	Low	High	SatsPath = identity only, no custody; wallet handles compliance